

FINAL PROJECT REPORT

다마고치 RPG

터미널 기반 육성·전투 복합 게임

TEAM

6CanDolt · 6조

MEMBERS

조광일 · 김태현 · 이현빈 · 정주연

DATE

2026년 06월

GITHUB

github.com/whsoo8232/tamagotchi-rpg

객체지향프로그래밍

목차

Table of Contents

1 프로젝트 개요

1.1 작품 주제 선정 배경

1.2 벤치마킹

1.3 개발 환경

2 시스템 설계

2.1 전체 아키텍처 (MVC 패턴)

2.2 파일 및 클래스 구성

2.3 클래스 다이어그램 및 상속 구조

2.4 사용자 경험(UX) 흐름

2.5 화면 구성 및 UI 설계

3 핵심 기능 구현 상세

3.1 게임 엔진 및 입출력 시스템

3.2 육성 시스템

3.3 훈련 시스템 (미니게임 3종)

3.4 전투 시스템

3.5 장비 및 아이템 시스템

4 객체지향 설계 원칙 적용

5 테스트 및 검증

6 팀원별 기여 역할

7 결론 및 기대효과

프로젝트 개요

Project Overview · Benchmarking · Development Environment

1.1 작품 주제 선정 배경

1990년대 출시된 휴대용 전자 펫 장난감 '다마고치(Tamagotchi)'는 가상의 생물을 돌보는 육성 게임으로 전 세계적으로 큰 인기를 끌었다. 이 프로젝트는 해당 콘셉트를 현대적인 **터미널 기반 RPG 형식**으로 재해석하여, 캐릭터를 직접 키우고 훈련시키며 전투에 참여하는 복합 장르 게임을 구현한다. C++ OOP 원리를 실제 게임 개발에 적용해보는 학습 목적도 겸한다.

OOP 학습

육성·전투·훈련·장비 등 다양한 기능을 모듈 단위로 나눠 OOP 설계를 자연스럽게 실습한다.

저수준 I/O 기술

터미널 환경에서 마우스·키보드 입력, ANSI 색상, 커서 제어 등 저수준 기술을 직접 구현·습득한다.

팀 협업

게임이라는 친숙한 도메인으로 팀 협업 동기를 부여하고 Git 기반 버전 관리 경험을 쌓는다.

1.2 벤치마킹

참고 작품	벤치마킹 포인트	적용 내용
반다이 다마고치 tamagotchi.fandom.com/wiki/Care	기본 육성 콘셉트	배고픔·행복도·청결도·건강 4종 스탯 실시간 관리 로직
디지털 월드 digimon_world_psx.fandom.com	목적형 훈련 + 스테이지 클리어	샌드백·달리기·반응속도 미니게임 및 5스테이지 해금 구조
언더테일 gamedeveloper.com/design	터미널 스타일 전투 UI	ASCII 아트 활용 전투 화면, 텍스트 기반 전략적 선택지

1.3 개발 환경

빌드 환경

언어: C++17
컴파일러: GCC (g++ -std=c++17)
빌드: 셸 스크립트 (build.sh)
OS: Linux WSL2 (Ubuntu)
IDE: VS Code + WSL 원격 연결

라이브러리 & 도구

termios.h — 터미널 raw mode
chrono — 논블로킹 타이머
memory — unique_ptr 스마트 포인터
vector, array — 컨테이너
Git / GitHub — 버전 관리

시스템 설계

Architecture · Class Diagram · UX Flow · UI Design

2.1 전체 아키텍처 (MVC 패턴)

Model

Character — 플레이어 데이터 (육성/전투 스탯, 장비, 아이템)

Monster — 몬스터 데이터 (스테이지별 HP/ATK/EVA)

Equipment / Item — 정적 티어·효과 테이블

View

Renderer — 정적 유틸리티 클래스

ANSI 색상 및 ASCII 아트 렌더링
UTF-8 / CJK 너비 계산
화면 레이아웃 헬퍼 함수

Controller

GameEngine — 게임 루프·화면 전환

InputHandler — 입력 이벤트 처리

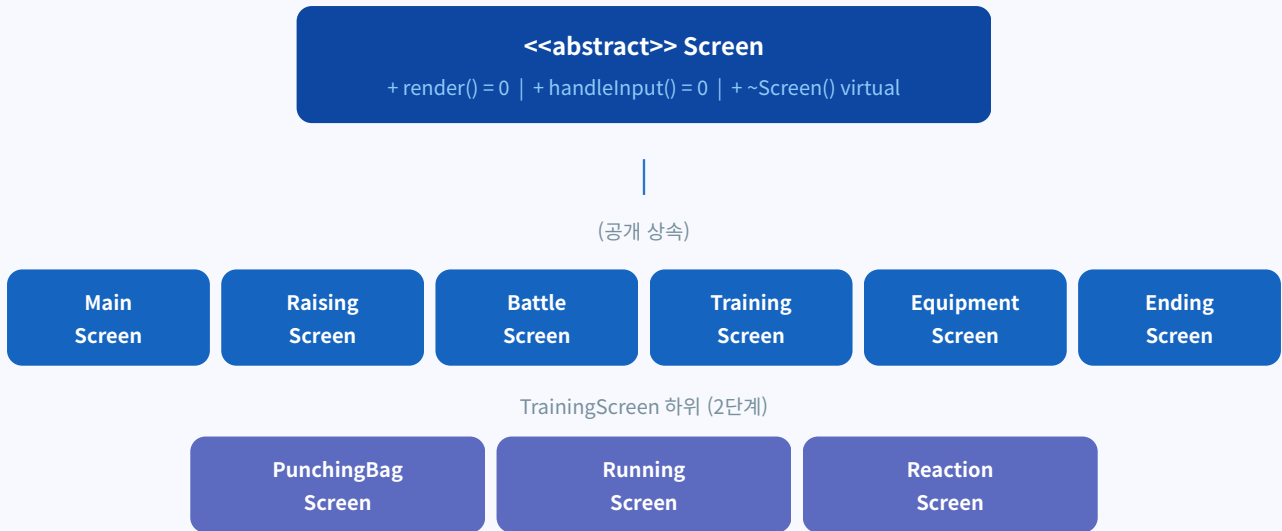
Screen (추상) — 공통 인터페이스
8개 **Screen** 서브클래스

2.2 파일 및 클래스 구성

경로	클래스	역할
src/models/Character.h/.cpp	Character	플레이어 모델 (스탯, 장비, 아이템)
src/models/Monster.h/.cpp	Monster	몬스터 모델 + 스테이지별 팩토리 메서드
src/models/Equipment.h · Item.h	—	검/갑옷 5티어 및 아이템 4종 정적 테이블
src/views/Renderer.h/.cpp	Renderer	ANSI 렌더링 정적 유틸리티 클래스
src/controllers/Screen.h	Screen (추상)	화면 공통 인터페이스 (순수 가상 함수 2개)
src/controllers/GameEngine.h/.cpp	GameEngine	게임 루프 및 화면 전환 관리
src/controllers/ InputHandler.h/.cpp	InputHandler	termios raw mode, 마우스 프로토콜 파싱
src/controllers/*Screen.h/.cpp	8개 화면 클래스	각 화면별 렌더링 + 입력 처리

2.3 클래스 다이어그램 및 상속 구조

SCREEN 상속 계층도 (INHERITANCE HIERARCHY)

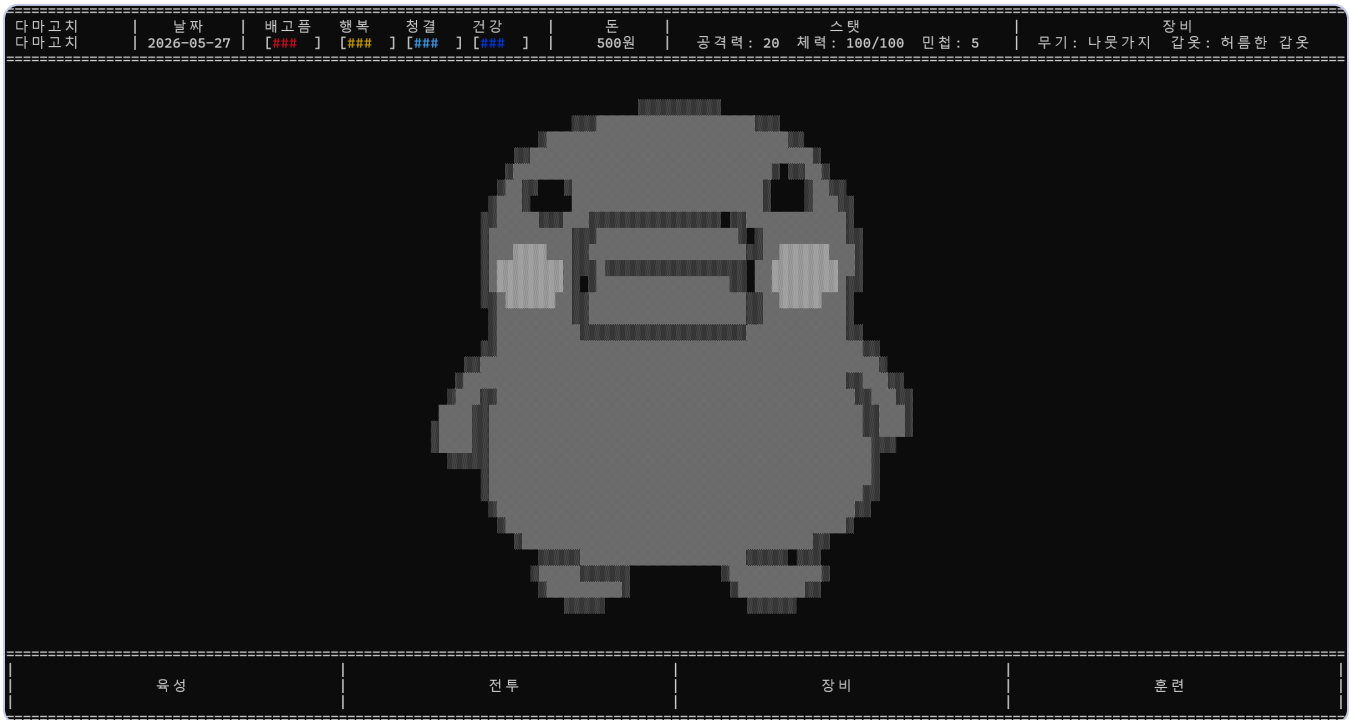


2.4 사용자 경험(UX) 흐름



2.5 화면 구성 및 UI 설계

터미널 화면은 **161컬럼 × 44행 고정 레이아웃**을 사용한다. 상단 스탯 바, 콘텐츠 영역(36행), 하단 버튼 바로 구성되며 마우스 클릭으로 모든 조작이 이루어진다.



실제 게임 화면 — 메인 화면(MainScreen) 및 각 서브 화면 UI 구성

화면	진입 방법	주요 인터페이스
MainScreen	시작 화면	구치파치 ASCII 아트 + 4개 메뉴 버튼
RaisingScreen	육성 버튼	먹이/목욕/놀이/치료, 1.2초 애니메이션
BattleScreen	전투 버튼	스테이지 선택 → HP바 + 가위바위보 손 아트
EquipmentScreen	장비 버튼	검/갑옷 강화, 아이템 상점
PunchingBagScreen	훈련 → 샌드백	10회 클릭 + 진행 바, 퍽! 피드백
RunningScreen	훈련 → 달리기	캐릭터가 결승선 방향으로 이동
ReactionScreen	훈련 → 반응속도	신호 색상 변화 + 반응시간(ms) 표시

게임 엔진 및 입출력 시스템

GameEngine · InputHandler · termios · Mouse Protocol

담당: 조광일

게임 루프 — GameEngine::run()

`unique_ptr<Screen>` 으로 현재 화면을 관리하며, 매 루프마다 논블로킹 입력 → 엔딩 판정 → 렌더링 순으로 동작한다.

```
void GameEngine::run() {
    while (running) {
        InputEvent event = inputHandler.pollEvent(); // 100ms 타임아웃
        bool hasInput = (event.type != InputType::NONE);
        if (hasInput) {
            currentScreen->handleInput(*this, player, event); // 다형성 디스패치
            if (player.isEndingCondition()) // 엔딩 조건 판정
                currentScreen = std::make_unique<EndingScreen>(et);
        }
        if (running && (hasInput || currentScreen->needsRedraw()))
            currentScreen->render(player);
        std::cout << std::flush;
    }
}
```

화면 전환 & 엔딩 조건

```
void GameEngine::changeScreen(std::unique_ptr<Screen> next) {
    currentScreen = std::move(next); // 이전 화면 자동 해제
}
engine.changeScreen(std::make_unique<BattleScreen>());
```

엔딩 조건 분기

배고픔=0 → STARVE | 행복=0 → RUNAWAY | 청결=0 → TRASH | 건강=0 → HOSPITAL | 5스테이지 클리어 → HERO

InputHandler — 터미널 입출력 핵심

raw mode (termios)

터미널을 즉시 전달 모드로 전환하여 키 입력을 실시간 처리한다. RAII 패턴으로 게임 종료 및 비정상 종료(Ctrl+C) 시에도 원래 터미널 설정으로 복원을 보장한다.

마우스 파싱 (xterm 프로토콜)

ESC 코드로 전달되는 마우스 이벤트를 직접 파싱하여 `InputEvent(type, button, x, y)` 로 정규화한다. 각 화면은 `event.x / btnWidth` 로 클릭된 버튼을 계산한다.

육성 시스템 & 훈련 시스템

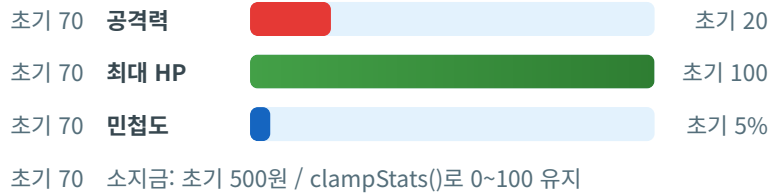
RaisingScreen · Character · TrainingScreen · MiniGames

3.2 육성 시스템 담당: 정주연

육성 스탯 (0~100)



전투 스탯 (훈련으로 증가)



액션	배고픔	행복	청결	건강	비용	애니메이션
밥주기	+20	+5	—	—	무료	1.2초
목욕하기	—	-5	+30	—	무료	1.2초
놀아주기	-10	+20	-10	—	무료	1.2초
치료하기	—	—	—	+30	100원	1.2초

전투 후 스탯 감소 — afterBattle()

전투 종료마다 health -1(고정) + {배고픔/행복/청결} 중 무작위 1개 -1이 적용된다. 전투를 반복하면 육성 관리가 필수적으로 요구되어 두 시스템이 유기적으로 연결된다.

3.3 훈련 시스템 (미니게임 3종) 담당: 이현빈

샌드백치기

목표: 10회 버튼 클릭
 효과: 공격력 +8
 피드백: 퍽! + 300ms 애니메이션
 진행 바: [|||||] [|||||]
 부작용: 배고픔·행복 -5

달리기

목표: 10걸음 버튼 클릭
 효과: 최대HP +20
 시각화: 캐릭터 → 결승선 이동
 $\text{이동량} = \text{runCount} \times \text{maxShift} / 10$
 부작용: 배고픔·행복 -5

반응속도

목표: 신호(노란 화면) 직후 클릭
 효과: 민첩 +1~6 (반응시간 기준)
 대기: 1~3초 랜덤, 0.6초 초과 시 실패
 논블로킹 std::chrono 타이머 사용
 부작용: 배고픔·행복 -5

전투 시스템

BattleScreen · Monster · Stage · Battle Logic

담당: 김태현

스테이지	몬스터	HP	공격력	회피율	보상
Stage 1	네시	80	15	5%	200원
Stage 2	파라사우롤로푸스	180	28	10%	400원
Stage 3	트리케라톱스	300	45	15%	600원
Stage 4	프테라노돈	450	65	20%	800원
Stage 5	티라노사우루스	650	90	25%	1,000원

전투 턴 흐름도



스테이지 해금 및 보상 규칙

이전 스테이지를 클리어해야 다음 스테이지가 해금된다. 전투 보상은 **stage × 200원**이며, 5스테이지 클리어 시 즉시 용사 엔딩 (HERO)으로 전환된다. 도망 시 보상 없이 스테이지 선택 화면으로 복귀한다.

장비 및 아이템 시스템

EquipmentScreen · Equipment · Item

담당: 김태현

검 강화 티어

티어	이름	공격 보너스	비용	성공률
0	나뭇가지	+0	200원	70%
1	나무 검	+10	350원	50%
2	단단한 검	+22	500원	30%
3	기사의 검	+38	700원	15%
4	용사의 검	+55	최대 단계	

갑옷 강화 티어

티어	이름	방어 감소	비용	성공률
0	허름한 갑옷	0	200원	70%
1	평범한 갑옷	-5	350원	50%
2	단단한 갑옷	-12	500원	30%
3	기사의 갑옷	-22	700원	15%
4	용사의 갑옷	-35	최대 단계	

강화 확률 시스템

강화 실패 시 **비용만 차감**되고 장비는 현재 티어를 유지한다. 높은 티어일수록 성공률이 낮아져 전략적 자원 관리가 요구된다.

전투 소모 아이템 — 4종 상세 설명

아이템 상점에서 구매하며 각 종류별 최대 2개 보유 가능. 매 턴 시작 전 1개를 선택하며 사용 즉시 소모된다.

힘의 포션 — 150원

효과: 승리 시 해당 턴 데미지를 **2배**로 증폭한다.
활용: 갑옷 방어력이 높거나 적 HP가 많이 남은 상황에서 효과적이다. 밧줄과 조합하면 회피 무시 + 2배 데미지를 동시에 적용할 수 있다.
조건: 가위바위보 승리 시에만 발동된다.

민첩의 포션 — 150원

효과: 패배 시 해당 턴의 회피율을 **2배**로 상승시킨다 (최대 100%).
활용: 높은 공격력의 적과 싸울 때 방어 수단으로 사용한다. 민첩이 50% 이상이면 확정 회피에 가까워진다.
조건: 가위바위보 패배 시에만 발동된다.

밧줄 — 200원

효과: 승리 시 몬스터의 회피를 **완전히 무시**하고 반드시 데미지를 입힌다.
활용: 회피율이 높은 후반 스테이지 적(프테라노돈 20%, 티라노사우루스 25%)에게 특히 유효하다.
조건: 가위바위보 승리 시에만 발동된다.

방패 — 200원

효과: 패배 시 몬스터의 공격을 **완전 차단**하여 받는 데미지를 0으로 만든다.
활용: 체력이 위험 수준에 있을 때 사용하면 생존에 결정적으로 도움이 된다.
조건: 가위바위보 패배 시에만 발동된다.

객체지향 설계 원칙 적용

OOP Principles — Abstraction · Inheritance · Polymorphism · Encapsulation

추상 클래스 (Abstract Class)

Screen 클래스가 `render()` 와 `handleInput()` 두 개의 순수 가상 함수를 선언한다. 모든 화면이 반드시 구현해야 할 계약 (contract)을 정의하며, Screen은 직접 인스턴스화 불가능하다.

상속 (Inheritance)

8개 화면 클래스 + 3개 훈련 서브클래스가 Screen을 공개 상속한다. TrainingScreen → PunchingBag / Running / ReactionScreen의 2단계 계층을 형성한다.

다형성 (Polymorphism)

`unique_ptr<Screen>` 하나로 모든 화면을 관리한다.
`currentScreen->render(player)` 호출 시 런타임에 실제 파생 클래스 메서드가 호출된다. if-else 분기 없이 `std::move` 로 화면이 전환된다.

캡슐화 (Encapsulation)

Character, Monster 등 모든 클래스의 멤버 변수는 `private`로 선언된다. 외부 접근은 public getter/setter를 통해서만 이루어지며, `clampStats()` 는 `private` 내부 유틸리티로만 사용된다.

정적 멤버 (Static Member)

Renderer의 모든 메서드와 색상 상수는 `static`으로 선언된다.
`Monster::generateStageMonster()` 는 `static` 팩토리 메서드이며, `SWORD_TABLE·ARMOR_TABLE·ITEM_TABLE`은 `static const array`로 구현된다.

스마트 포인터 (Smart Pointer)

`unique_ptr<Screen>` 으로 화면을 관리한다. 화면 전환 시 `std::move`로 소유권이 이전되며 이전 화면은 자동 해제된다. 가상 소멸자와 함께 메모리 누수 없이 동작한다.

상태 머신 (State Machine)

BattleState, RCState, PBState, RNState, EquipState 등 enum class로 각 화면의 내부 상태를 타입 안전하게 관리한다. 상태 전환은 각 `handleInput` 내에서 명시적으로 이루어진다.

RAII 패턴

InputHandler 생성자에서 `termios raw mode`를 활성화하고 소멸자에서 복원한다. Ctrl+C 등 비정상 종료 시에도 정적 시그널 핸들러가 터미널 상태를 안전하게 복원함을 보장한다.

테스트 및 검증

Manual Testing · Test Results

별도의 자동화 테스트 프레임워크는 사용하지 않았으며, 빌드 후 직접 실행하는 수동 테스트 방식을 채택하였다.

테스트 항목	테스트 방법	결과
빌드 성공 여부	bash build.sh 실행 후 오류 확인	정상
화면 전환	각 메뉴 버튼 클릭 및 키보드 입력	정상
캐릭터 스탯 변화	먹이/목욕/놀이/치료 후 수치 확인	정상
전투 시스템	5스테이지 전체 가위바위보 전투 진행	정상
훈련 미니게임 3종	각 게임 플로우 후 스탯 획득 확인	정상
장비 강화 성공·실패	검/갑옷 강화 케이스별 확인	정상
아이템 구매 및 전투 사용	4종 아이템 구매 후 전투 중 사용	정상
게임 오버 조건 (4종)	스탯 극한까지 낮춰 종료 트리거	정상
엔딩 5종	각 조건 달성 후 엔딩 화면 확인	정상
UTF-8 / CJK 렌더링	한글·이모지 포함 문자열 정렬 확인	정상

팀원별 기여 역할

Team Contribution by Member

조광일

게임 엔진 · 입출력 시스템

GameEngine InputHandler Screen.h

termios 기반 raw mode 입력 처리 시스템 설계 및 구현

xterm 마우스 프로토콜 파싱 파서 직접 구현

GameEngine 게임 루프 및 화면 전환 구조 설계

unique_ptr 기반 다형성 화면 관리 시스템 구현

stdout 버퍼링으로 화면 깜빡임 방지 최적화

비정상 종료 시 터미널 복원 시그널 핸들러 구현

추상 클래스 다형성 termios RAI

김태현

전투 · 장비 로직 설계 및 구현

BattleScreen EquipmentScreen Monster

5단계 스테이지별 몬스터 설계 및 팩토리 메서드 구현

가위바위보 기반 전투 알고리즘 설계 및 구현

전투 아이템 효과 적용 로직 (4종) 구현

검/갑옷 5티어 강화 확률 시스템 설계 및 구현

전투 화면 3열 레이아웃 및 HP 바 시각화 구현

스테이지 잠금 해제 시스템 구현

상속·오버라이딩 상태 머신 팩토리 메서드

이현빈

훈련 로직 설계 및 구현 · 캐릭터 디자인

TrainingScreen PunchingBag Running Reaction

샌드백 치기 미니게임 설계 및 구현 (공격력 +8)

달리기 미니게임 (캐릭터 이동 애니메이션, 체력 +20) 구현

반응속도 측정 미니게임 (민첩 +1~6) 논블로킹 구현

구치파치 및 몬스터 캐릭터 ASCII 아트 디자인

훈련 미니게임 진행 바 시각화 구현

상태 머신 std::chrono ASCII 아트 디자인

정주연

육성 로직 설계 및 구현

Character RaisingScreen Renderer

캐릭터 육성 스탯 구조 (4종) 및 전투 스탯 설계

밥주기/목욕/놀아주기/치료 액션 및 스탯 변화 구현

전투 후 스탯 감소(afterBattle) 시스템 구현

Renderer 정적 유틸리티 클래스 설계 및 구현

UTF-8+CJK 문자 너비 계산(getDisplayWidth) 구현

ANSI 색상 HP바 및 161컬럼 레이아웃 시스템 설계

캡슐화 정적 클래스 UTF-8 계산

결론 및 기대효과

Conclusion & Expected Effects

구현 완료 기능 요약

기능	구현 상태
캐릭터 육성 시스템 (먹이/목욕/놀이/치료 + 애니메이션)	완료
전투 시스템 (5스테이지, 가위바위보, 몬스터 회피·방어)	완료
훈련 미니게임 3종 (샌드백·달리기·반응속도)	완료
장비 강화 시스템 (검/갑옷 각 5티어, 확률 강화)	완료
아이템 시스템 (4종, 상점 구매 및 전투 중 사용)	완료
마우스 클릭 통합 입력 시스템 (xterm 프로토콜)	완료
ANSI 색상 및 ASCII 아트 렌더링	완료
엔딩 시스템 5종 (용사·굶주림·도망·쓰레기·병원)	완료
UTF-8 / CJK 렌더링 정렬	완료

기대효과 및 활용방안

추상 클래스 · 상속 · 다형성

추상 클래스 Screen을 중심으로 8개 구체 화면 클래스를 설계·구현하여, 코드 중복 없이 다형성으로 게임 루프를 운영하는 구조를 완성하였다.

저수준 시스템 프로그래밍

termios, ANSI Escape Code, xterm 마우스 프로토콜 등 OS·터미널 수준의 저수준 기술을 직접 구현하였다.

논블로킹 타이밍 게임

std::chrono 기반 논블로킹 상태 머신으로 3종의 반응속도 미니게임을 구현하여 실시간 인터랙션을 달성하였다.

팀 협업 · Git 경험

Git을 통한 버전 관리 및 코드 리뷰 경험을 습득하였으며, 모듈화된 구조로 각자 독립적인 기능 개발이 가능하였다.

객체지향 교육 레퍼런스

복잡한 상태 관리와 OOP 설계의 정석을 담아, 객체지향 프로그래밍 교육용 레퍼런스나 로직 설계의 모범 사례로 활용 가능하다.

저사양 환경 적용

터미널만 있으면 실행 가능하여 라즈베리파이, 서버 환경 등 GUI가 없는 환경에서도 동작한다. TUI 기반 게임의 참고 아키텍처로 활용 가능하다.